What YV Has Done

What YV Has Done

An evidence-based account of the MoE-FFD baseline implementation, Kaggle training, evaluation, and handover preparation.

Project: DA6021 | Model: MoE-FFD | Dataset: FaceForensics++ C23 | Report date: 9 November 2026

This report records the process and results provided from the Kaggle notebook and current DA6021 project notes. Kaggle outputs were supplied in chat and were not independently re-opened from the account. Treat those outputs as the evidence source; preserve the notebook, logs, JSON, and checkpoint for an auditable handover.

1. Project goal and baseline scope

The agreed order of work was to implement and establish the authors· baseline model first, then let the next team stage explore a modest improvement. The model is MoE-FFD (Mixture of Experts for Generalized and Parameter-Efficient Face Forgery Detection), based on a ViT-B/16 backbone and parameter-efficient expert components including LoRA and Convpass adapters, with gating and an MoE balancing loss.

The Kaggle work trained on the prepared FaceForensics++ C23 frame dataset. The reported split folders were train, valid, and test, each containing original, Deepfakes, Face2Face, FaceSwap, and NueralTextures categories. The loader·s spelling NueralTextures was retained. This report calls the result a five-epoch baseline; the paper·s reported schedule was longer, so this is not claimed as a complete paper reproduction.

The local DA6021 project includes the authors· implementation and documentation, but the completed training evidence comes from the separate Kaggle runtime. Ensure the exact patched Kaggle source is included in the handover rather than assuming the local checkout is identical.

2. Code and environment preparation

The working source was placed in /kaggle/working/repo. The notebook checked Python imports for torch, OpenCV, dataset, and ViT_MoE, then checked CUDA availability and loaded sample original and manipulated images. The intended image check transformed a two-image batch and asserted tensor shape (2, 3, 224, 224) and finite values.

The Kaggle environment used Python 3.13 and a modern timm version, while the upstream code followed an older timm API. Import initially failed because model metadata used hf_hub, which the installed PretrainedCfg did not accept; the compatible field was hf_hub_id. The old model builder call also supplied default_cfg and pretrained_custom_load in a form incompatible with the installed builder, producing an unexpected default_cfg argument error. The code was patched in Kaggle to adapt these arguments. Registry overwrite and deprecated import messages appeared afterward; they were warnings, while the initial TypeErrors were actual blockers.

The optimizer parameter groups were also amended to set explicit learning rates. The recorded intended rates were 1e-4 for gate parameters and 3e-5 for other trainable parameters. A backup of train.py was created before this change. The exact patched files should be kept with the checkpoint because an unpatched local checkout may not import or train in the same runtime.

Before training, a forward/backward smoke check was run using one original and one Deepfakes image. It reported classification loss 0.450300, MoE balancing loss 6.360782, total loss 6.811081. Gradients were finite; nonzero-gradient parameter tensors were reported for gates 48/48, LoRA 84/168, adapters 46/360, and other 26/26. The notebook explicitly cleared gradients and did not call optimizer.step(), so this check did not update weights.

3. Training process

The model had 101,290,946 total parameters. The run log reported 15,510,722 trainable parameters: 14,579,712 associated with LoRA, 911,040 with adapters, and 1,538 in the head. The reported batch size was 32, 20 training frames, two workers, and 3,600 training iterations per epoch.

Training did not finish in a single uninterrupted kernel session. A kernel interruption required a restart and the team rechecked model imports, sample transforms, forward/backward behavior, and checkpoint state. Training was then resumed from saved checkpoints in stages. The successful final continuation started with ·Resuming after epoch 3· and best validation AUC 0.986390, then completed epochs 4 and 5. This staged resume avoided repeating the already completed epochs.

The final log showed training progress through 3,600 iterations in epoch 4 and 3,600 iterations in epoch 5, followed by evaluation. The completion message stated that the task for /kaggle/working/models/baseline was completed; the reported total runtime for that continuation was 2 hours 9 minutes. PyTorch AMP and CUDA tensor constructor deprecation warnings appeared; no evidence supplied indicates they stopped the run.

4. Validation results

Epoch 4 validation: video accuracy 92.71%, video AUC 0.9834, video EER 6.43%; frame accuracy 89.04%, frame AUC 0.9602, frame EER 10.32%.

Epoch 5 validation: video accuracy 93.29%, video AUC 0.9873, video EER 5.54%; frame accuracy 90.41%, frame AUC 0.9660, frame EER 8.88%.

The startup log for the last continuation recorded the best validation AUC from the preceding checkpoint as 0.986390. Epoch 5·s reported validation AUC was 0.9873. Validation metrics were used to assess the run; the reported held-out test evaluation was a separate later step.

5. Checkpoint and integrity

The saved file was /kaggle/working/models/baseline/models_params_4.tar, approximately 530.1 MB. The index is zero-based: models_params_0.tar corresponds to completed epoch 1, so models_params_4.tar corresponds to completed epoch 5. This naming detail should be made explicit to recipients.

The file was copied to /kaggle/working/models_params_4.tar to make it easier to download. A later attempt to use notebook FileLink returned a 404, so the file was copied into the top-level working directory. Because the full file was difficult to download, a splitting cell made six additional .part files and a SHA256 file. The user subsequently downloaded the original full TAR and computed its SHA256 locally. It matched the Kaggle-reported digest: 49432ca97fe6a07d72c9f02afcf1aa3bac8b386f91c6302ea152711ac399d6ad. Thus the original checkpoint was not broken into fragments; no reconstruction is necessary. The part files are duplicate download aids and can be deleted once no longer needed.

The checkpoint can be loaded by the compatible model code for evaluation and inference, and may contain optimizer and other resume state depending on the training save routine. It is not a standalone executable and does not include the dataset, face detection, frame extraction, or inference application.

6. Held-out test evaluation

The repository inspection showed eval.py and eval_csv.py. The provided eval.py accepts --model_path, --test_root, --dataset, --frames, --num_workers, --device, and --out_json. Its test routine loads model_state_dict when the checkpoint is a training TAR. The selected invocation evaluated /kaggle/working/data/c23/test as FFPP with 20 frames per video, GPU 0, and two workers.

The test run completed 700 video items in about 207 seconds of evaluation time, with reported total runtime around 3 minutes. The test result was: video accuracy 95.00%, AUC 0.9852423469, EER 4.1071%; frame accuracy 90.2786%, AUC 0.9639870, EER 9.3839%. Rounded for reporting: video accuracy 95.00%, AUC 98.52%, EER 4.11%; frame accuracy 90.28%, AUC 96.40%, EER 9.38%.

The evaluation script averages frame fake probabilities to form a video score and calculates metrics at threshold 0.5. The test JSON records the model path, dataset, test root, frame count, number of videos, and frame/video metrics. Keep that JSON and the console log together. These are results for the supplied split and protocol, not a guarantee for other sources or real-world use.

7. Handover preparation and sharing

A small handover ZIP was generated in Kaggle at /kaggle/working/baseline_epoch5_handover.zip, reported at about 0.12 MB. It was intended to contain source, environment text, instructions, and result files while excluding the large model and dataset. The archive must be inspected after download to ensure it contains the exact successful patched Kaggle code and ffpp_test_epoch5.json. A notebook export should also be saved as .ipynb.

The local DA6021 folder contains the MoE-FFD checkout, a Kaggle notebook, project PDFs and notes. The user has the checkpoint in C:\Users\sreey\Downloads\models_params_4.tar and its hash is verified. A practical team handover is to place code, documentation, notebook, environment, logs, and small JSON results in a GitHub repository; attach the 530 MB checkpoint to a GitHub Release or approved shared artifact, not ordinary Git history. Give recipients the checksum and ensure they have permission to access the dataset. Dataset files should not be redistributed without checking the provider·s terms.

Teammates who want to run the model should use a copy of the shared Kaggle notebook with GPU enabled, attach the same dataset and obtain the checkpoint, run setup and path cells, and first execute evaluation without training. Only after matching the baseline should they start modifications. This separates baseline reproduction from improvement work.

8. Suggested supporting evidence

Before ending the Kaggle session, retain the exported notebook, text training logs, evaluation log, JSON metrics, checkpoint, checksum, and a screenshot of the notebook output. Helpful screenshots are: the epoch-5 validation result and completion line; the test result showing 700 videos and final video/frame metrics; Kaggle·s file panel showing model and result artifacts; and the notebook title plus executed evaluation cell.

Screenshots should have captions and be used as supplementary evidence. The JSON and raw logs are better primary evidence because they are searchable and machine-readable. Do not expose access tokens, private account data, gated data URLs, or unnecessary face images. No Kaggle screenshots were supplied for this report; the process above is a capture checklist, not a claim that images have already been saved.

9. Possible next improvements

First audit the data and splits. Verify train/valid/test video identities are disjoint and that manipulated clips and their source originals cannot cross splits. Check duplicates, class counts, extraction failures, and the precise sampling policy. High scores do not rule out leakage or dataset-specific shortcuts.

Then establish controlled experiments. A longer run toward the paper·s 20 epochs is a possible follow-up, with correct optimizer/scheduler/scaler resume and validation-based stopping. Test several fixed seeds and report spread. Try modest learning-rate or schedule changes one at a time. Keep the baseline checkpoint immutable.

Potential architecture experiments include LoRA rank, Convpass bottleneck width, number of experts, top-k routing, or selectively unfreezing the final ViT block. Potential objective experiments include the MoE balancing-loss coefficient. Log expert usage and routing entropy to detect collapse. Potential data experiments include realistic compression, blur, color and resize perturbations. Potential inference experiments include alternative frame aggregation, selected on validation only.

For generalization, evaluate approved external datasets such as Celeb-DF-v2 or DFDC Preview, with each dataset and preprocessing reported separately. Test compression, resolution changes, unseen manipulation methods, and capture conditions. Inspect false positives and false negatives, and measure inference latency, memory and throughput. Do not repeatedly tune against the test set. Report the chosen improvement on the held-out test only after validation-based selection, with the same protocol as baseline.

For each run record the source revision, hypothesis, seed, data version, initialization checkpoint hash, parameters, runtime, validation metrics, selected checkpoint, and final test metrics. Report unsuccessful runs and avoid cherry-picking seeds. Describe the existing result as a five-epoch baseline, not a perfect model or complete paper reproduction.

10. What remains to verify

The current report is based on outputs pasted from Kaggle and project files in DA6021. Before formal submission, inspect the downloaded ZIP and verify it includes the exact patched source and test JSON. Confirm the exported notebook, logs, and environment snapshot are safely stored locally. Verify official split provenance and identity separation. Confirm licensing and redistribution terms for source, pretrained weights, and datasets. A matching SHA256 establishes that the downloaded TAR matches the reported digest; it does not establish data provenance or scientific validity.

The provided local README and continuation note predate the completed five-epoch Kaggle run and contain older or generic commands. Prefer the exact successful Kaggle notebook/source snapshot and the command in the JSON/log over stale local instructions. Update the project README after confirming the packaged files.

Summary of reported outcomes

Completed: five epochs of baseline training; epoch-5 checkpoint downloaded and SHA256 verified; FF++ C23 held-out test evaluation on 700 videos at 20 frames per video; result output and handover preparation.

Final reported test: video AUC 98.52%, accuracy 95.00%, EER 4.11%; frame AUC 96.40%, accuracy 90.28%, EER 9.38%.

Next: archive the exact working code and evidence, share checkpoint as a release/shared artifact, have a teammate reproduce evaluation, audit split integrity, then pursue one controlled validation-selected improvement.